МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГБОУ ВО «Воронежский Государственный Технический Университет»
Кафедра Автоматизированных и Вычислительных систем

СОГЛАСОВАНО
Заказчик
Сафронов В. В. / _____
«1» сентября 2026 г.

УТВЕРЖДАЮ
Руководитель проекта
Машнев М. / _____
«1» сентября 2026 г.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на разработку системы управления строительными дефектами и рекламациями
«DefectTrack» (Defect Management System)

Выполнил: студент группы бИВТ-231
ФИО: **Машнев М.**
Преподаватель: **Сафронов В. В.**

Воронеж 2026

ВВЕДЕНИЕ

Современные девелоперские компании, генеральные подрядчики и управляющие организации при сдаче и гарантийной эксплуатации объектов капитального строительства (жилых комплексов, коммерческой недвижимости) сталкиваются с непрерывным потоком рекламаций и замечаний. Строительные дефекты — от нарушений герметичности стеклопакетов и сколов плитки до аварийных протечек стояков горячего водоснабжения и замыканий электропроводки — требуют оперативной регистрации, верификации, объективной оценки срочности и диспетчеризации ответственным подрядным организациям.

В настоящее время во многих управляющих и строительных компаниях фиксация замечаний жильцов и арендаторов ведётся разрозненно: через бумажные журналы, чаты мессенджеров, электронную почту или общие электронные таблицы. Подобный подход порождает критические системные проблемы: - субъективность и задержки ручной оценки критичности дефектов; - затягивание нормативных сроков устранения аварийных недочётов; - отсутствие прозрачного трекинга для заявителя (жильца/дольщика); - отсутствие централизованного архива фото- и видеофиксации недостатков; - невозможность получения оперативной сквозной аналитики по повторяющимся дефектам и проблемным строительным объектам.

Настоящее техническое задание определяет комплекс требований к созданию клиент-серверной web-системы **«DefectTrack» (Defect Management System)**, обеспечивающей автоматизированный приём рекламаций от заявителей, автоматический многофакторный расчёт приоритета заявок, ведение реестра строительных объектов, диспетчеризацию работы инженеров-менеджеров и визуализацию аналитических показателей в реальном времени.

Назначение системы

Система предназначена для комплексного автоматизированного управления жизненным циклом рекламаций на строительные дефекты, объективного алгоритмического расчёта критичности и приоритета выполнения работ, централизованного хранения фото/видеоматериалов дефектов и мониторинга качества возводимых и сданных объектов строительства.

Цели создания системы

1. **Автоматизация приёма и регистрации обращений:** обеспечение удобного круглосуточного канала подачи заявок с фото- и видеофиксацией дефектов без обязательной предварительной регистрации заявителя.
2. **Алгоритмический расчёт приоритета (Scoring Engine):** исключение субъективного человеческого фактора при распределении ресурсов за счёт автоматического многофакторного скоринга срочности дефекта (1–10 баллов).

3. **Управление жизненным циклом заявок:** регламентация переходов между статусами («новый» → «в работе» → «выполнено» / «отклонено») с фиксацией точных таймстемпов, ответственных и мотивированных причин.
4. **Контроль строительных объектов:** ведение единого паспорта зданий и сооружений с привязкой дефектов и подсчётом накопительной статистики замечаний.
5. **Аналитика и отчётность:** предоставление руководству и менеджерам наглядного интерактивного дашборда для выявления системных проблем качества работ подрядчиков.
6. **Сокращение времени реагирования:** минимизация времени от момента подачи рекламации до взятия в работу дежурными специалистами.

Термины и сокращения

Таблица 1 – Термины и сокращения

Термин	Расшифровка
ЖК	Жилой комплекс / объект строительства
Дефект (рекламация)	Несоответствие строительного конструктива, инженерных сетей или отделки проектным нормам и СНиП
Приоритет (скоринг)	Численная оценка срочности устранения дефекта в диапазоне от 1 (низкий) до 10 (критический), рассчитываемая алгоритмически
SPA	Single Page Application — одностраничное web-приложение
FSD	Feature-Sliced Design — архитектурная методология организации фронтенд-кода
MUI	Material-UI — библиотека интерфейсных компонентов для React
MobX	Реактивная библиотека управления состоянием приложения
JWT	JSON Web Token — компактный защищённый токен аутентификации
RBAC	Role-Based Access Control — ролевая модель разграничения доступа
MIME type	Стандарт описания типов данных в сети Интернет (Multipurpose Internet Mail Extensions)
ИИ-ассистент	Программное средство на основе большой языковой модели (LLM), привлекаемое для ускорения разработки и тестирования

1. ОСНОВАНИЕ ДЛЯ РАЗРАБОТКИ

1.1. Основание и наименование

Основанием для разработки является учебный проект по проектированию программных систем и автоматизации процессов управления строительными гарантийными обязательствами.

- **Полное наименование:** Система автоматизированного приёма, оценки и управления рекламациями на строительные дефекты «DefectTrack».
- **Краткое наименование:** Defect Management System (DefectTrack).

1.2. Участники проекта

Таблица 2 – Участники проекта

Роль	Описание
Заказчик	Строительно-девелоперская компания / управляющая организация (наименование уточняется)
Исполнитель	Команда разработки в составе студента группы бИВТ-231 Машнева М. под руководством преподавателя Сафронова В. В.
Источник финансирования	Учебный грант / бюджет научно-исследовательских и опытно-конструкторских работ кафедры АВС
Срок разработки	16 недель (около 4 месяцев), см. раздел 8

1.3. Объект автоматизации

Объектом автоматизации является технологический и организационный процесс диспетчеризации заявок по строительным дефектам на эксплуатируемых и строящихся объектах компании. Иерархия объектов представлена следующим образом:

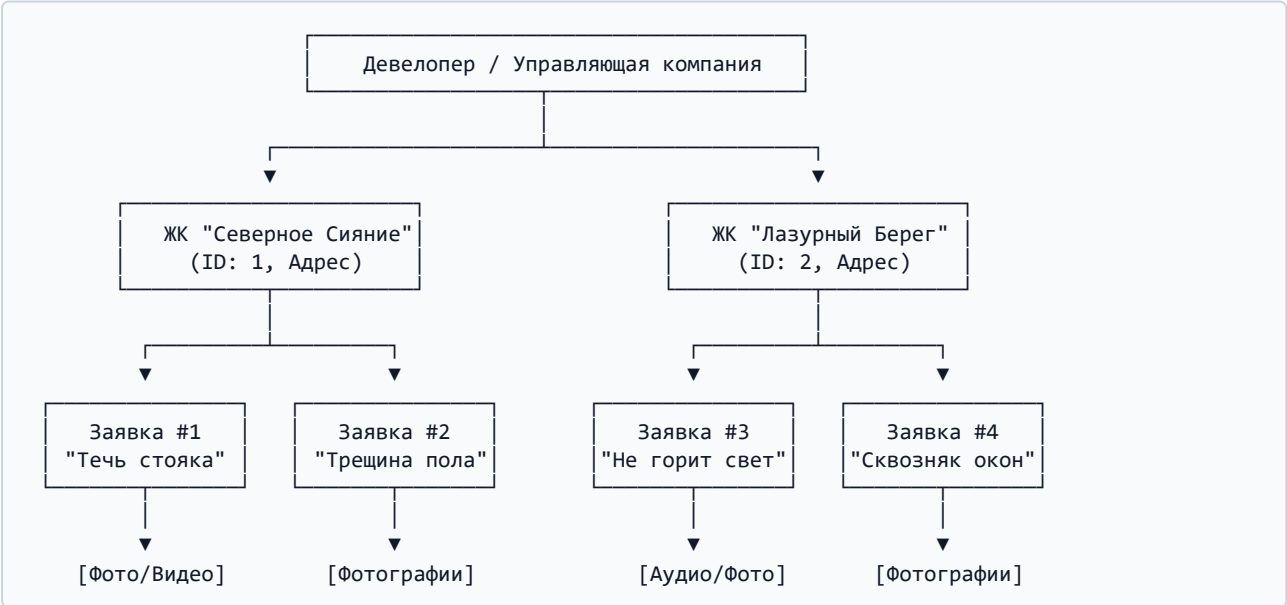


Рисунок 1 – Структурная схема объектов автоматизации

Каждое здание обладает паспортом (наименование, адрес, дата ввода в эксплуатацию, суммарное число поданных рекламаций). К зданию привязываются входящие заявки от жильцов с указанием контактного email, описания проблемы, категории дефекта и вложенных файлов.

---

2. ТРЕБОВАНИЯ К ПРОГРАММЕ

2.1. Общие требования

- Система реализуется как клиент-серверное web-приложение (SPA + REST API);
- Серверная часть (backend) разрабатывается на компилируемом высокопроизводительном языке **Go (Golang)** версии не ниже 1.24 с использованием фреймворка **Gin**;
- Клиентская часть (frontend) разрабатывается на базе **React 19** с использованием языка **TypeScript**, компонентной библиотеки **Material-UI (MUI)** и менеджера реактивного состояния **MobX**;
- Архитектура фронтенда базируется на принципах **Feature-Sliced Design (FSD)**;
- Система хранения данных предусматривает работу с **PostgreSQL 16** (для развёртывания в Docker/production) и встроенным **SQLite** (для автономного локального тестирования и разработки);
- Развёртывание и изолированный запуск серверных служб реализуются средствами **Docker** и **Docker Compose**.

2.2. Требования к производительности

Таблица 3 – Показатели производительности

Показатель	Требование
Количество хранимых заявок	Не менее 50 000 без деградации времени выборок
Число одновременно работающих пользователей	До 500 активных пользователей (жильцы + менеджеры)
Время отклика REST API	Не более 300 мс (95-й процентиль) для стандартных операций CRUD
Время расчёта алгоритма скоринга приоритета	Не более 50 мс на одну заявку
Максимальный размер загружаемого файла	До 25 МБ на один файл (фотографии высокого разрешения, видеофрагменты)
Время инициализации локальной БД (миграции + сиды)	Не более 5 секунд при холодном старте сервиса

2.3. Требования к надёжности

- Коэффициент доступности сервиса — не ниже 99,5% в месяц;
- Сохранение целостности транзакций при создании заявок с одновременным связыванием загруженных файлов;
- Корректная обработка разрыва сетевого соединения при отправке формы на клиенте с информированием заявителя;
- Автоматический перезапуск сервисов в Docker контейнерах при аварийных падениях (`restart: unless-stopped`);
- Резервное копирование базы данных PostgreSQL по регламенту не реже 1 раза в сутки.

2.4. Требования к информационной безопасности

- Защита административных и менеджерских разделов с помощью механизма **JWT** (JSON Web Token, алгоритм HMAC-SHA256) со сроком жизни токена (TTL) 24 часа;
- Хранение паролей учётных записей исключительно в хэшированном виде с применением криптостойкого алгоритма **bcrypt** (cost factor не менее 10);
- Разграничение ролей доступа (RBAC): 1. *Анонимный заявитель (Гость)* — подача заявки, загрузка файлов, просмотр статуса собственной заявки по идентификатору; 2. *Менеджер объекта* — просмотр всех заявок, фильтрация, перевод по статусам («в работе», «выполнено», «отклонено»), доступ к аналитике; 3. *Администратор* — полный доступ, включая управление объектами, категориями и аккаунтами сотрудников;
- Защита от типовых web-угроз:
- экранирование SQL-параметров через prepared statements (защита от SQL Injection);
- строгая валидация MIME-типов и расширений загружаемых файлов (защита от Remote Code Execution);
- защита от Path Traversal при отдаче файлов из локального каталога `uploads/` ;
- настройка CORS-политики для ограничения неавторизованных внешних источников.

2.5. Требования к совместимости и интерфейсам

- **Поддерживаемые браузеры:** актуальные версии Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge, Яндекс.Браузер;
- **Адаптивность:** корректное отображение на мониторах ПК (от 1280px), планшетах и смартфонах (от 360px);
- **Протокол взаимодействия:** REST API поверх HTTP/1.1 (HTTPS в production), формат сериализации данных — JSON;

- **Формат передачи файлов:** `multipart/form-data`.

---

3. ТРЕБОВАНИЯ К ФУНКЦИОНАЛЬНЫМ ХАРАКТЕРИСТИКАМ

3.1. Модуль приёма и трекинга заявок (Публичный контур)

Система должна обеспечивать: 1. Заполнение формы подачи заявки без обязательной аутентификации: - выбор строительного объекта из выпадающего справочника зарегистрированных ЖК; - выбор категории дефекта (водоснабжение, электрика, отделка, окна/двери и др.); - субъективная оценка критичности заявителем (низкая, средняя, высокая, аварийная); - ввод контактного email для обратной связи; - подробное текстовое описание проблемы; - прикрепление одного или нескольких файлов фото- и видеофиксации дефекта. 2. Проверку корректности введённых данных перед отправкой на клиенте и сервере; 3. Выдачу пользователю уникального идентификатора заявки (ID) для последующего отслеживания прогресса; 4. Экран трекинга заявки по её номеру, отображающий текущий статус, дату приёма, назначенный приоритет, историю изменений и прикреплённые фотоматериалы.

3.2. Алгоритмический модуль многофакторного скоринга приоритета

При создании каждой заявки серверный компонент **PriorityService** выполняет детерминированный расчёт итогового скоринга (приоритета) в диапазоне от **1** (минимальный) до **10** (критический / наивысший).

Расчёт итогового приоритета выполняется по формуле:

$$Priority = \max(1, \min(10, S_{общ})) \quad (1)$$

где $S_{общ}$ — суммарный показатель критичности заявки с учётом всех факторов:

$$S_{общ} = S_{баз} + S_{кат} + S_{опис} + S_{медиа} + S_{время} + S_{повтор} + S_{объект} \quad (2)$$

Составляющие суммарного скоринга определяются следующими правилами:

1. **Базовый балл критичности ($S_{баз}$):** исходная оценка заявителя в форме ($S_{баз} \in [1..4]$): - «Низкая» — 1 балл; - «Средняя» — 2 балла; - «Высокая» — 3 балла; - «Аварийная» — 4 балла.
2. **Вес категории дефекта ($S_{кат}$):** - Инженерные сети (водоснабжение, канализация, отопление, газ, электрика) — повышенный вес (+2 ... +3 балла); - Отделочные и косметические дефекты (штукатурка, плинтусы, сколы) — базовый вес (+1 балл).

3. **Глубина и детализация описания ($S_{опис}$):** - Длина текста ≥ 200 символов: +2 балла (детальное изложение проблемы); - Длина текста от 50 до 199 символов: +1 балл; - Менее 50 символов: 0 баллов.
4. **Наличие доказательной медиабазы ($S_{медиа}$):** - Прикреплено ≥ 3 файлов: +2 балла; - Прикреплено 1–2 файла: +1 балл; - Наличие хотя бы одного видеофайла (MIME-тип `video/*`): дополнительно +1 балл (суммарно до +3 баллов, подтверждает динамический характер дефекта, например течь или искрение).
5. **Фактор времени регистрации ($S_{время}$):** - Подача заявки в ночные / нерабочие часы (с 22:00 до 06:00): +2 балла (высокая вероятность срочной аварии); - Дневное время (с 06:00 до 22:00): 0 баллов.
6. **История повторяемости дефектов на объекте ($S_{повтор}$):** - Наличие открытых заявок в той же категории дефекта на данном объекте: +2 балла (признак общедомовой неполадки).
7. **Аварийная нагрузка объекта ($S_{объект}$):** - Объект является лидером по накопленному числу обращений среди всех зданий: +1 балл.

3.3. Модуль управления объектами строительства (Зданиями)

Система должна обеспечивать: - Ведение реестра строительных комплексов и зданий (название, физический адрес, дата регистрации объекта); - Автоматический подсчёт счётчика активных и суммарных рекламаций по каждому объекту (`numberApplications`); - Поиск и фильтрацию зданий по ключевым словам в названии и адресе; - Добавление, редактирование и архивирование объектов авторизованными менеджерами.

3.4. Модуль жизненного цикла и обработки заявок (Контур менеджера)

Поддерживается строгий жизненный цикл заявки со следующими состояниями: - **«новый»** (исходное состояние после отправки формы заявителем); - **«в работе»** (принята инженером к исполнению, фиксируется дата начала `dateInProgress`); - **«выполнено»** (дефект устранён подрядчиком, фиксируется дата завершения `dateResult`); - **«отклонено»** (заявка признана дубликатом или негарантийным случаем, с обязательным заполнением текстового обоснования `reason`).

Менеджер имеет возможность: - Просматривать реестр заявок в виде интерактивной таблицы с сортировкой по приоритету, дате и объекту; - Фильтровать заявки по статусу, уровню критичности, категории и конкретному зданию; - Просматривать детальную карточку заявки с галереей прикреплённых изображений; - Выполнять перевод заявки в следующий статус с валидацией обязательных полей.

3.5. Модуль аналитики и мониторинга

Модуль предоставляет агрегированные статистические данные: - Общее количество зарегистрированных рекламаций за всё время; - Распределение заявок по статусам («новый»,

«в работе», «выполнено», «отклонено»); - Топ проблемных строительных объектов по числу открытых дефектов; - Распределение дефектов по категориям инженерных систем; - Среднее время реагирования и закрытия заявок.

3.6. Состав экранов web-интерфейса

Таблица 5 – Экраны системы

Экран	Назначение и состав элементов
Главная / Подача заявки	Публичная форма: выбор ЖК, категории, выбор критичности, email, текстовая область описания, Drag-and-Drop область прикрепления файлов, кнопка «Отправить»
Трекер статуса заявки	Поле ввода ID заявки, информационный таймлайн статуса, дата подачи, приоритет, прикреплённые фото и комментарий решения
Авторизация персонала	Форма входа менеджера/администратора: поля логина и пароля, обработка ошибок авторизации
Реестр заявок (Менеджер)	Таблица заявок (MUI DataGrid) с пагинацией, фильтрами по статусам и ЖК, цветовой подсветкой приоритетов (1–3 зелёный, 4–7 жёлтый, 8–10 красный), кнопка смены статуса
Карточка заявки	Детальное отображение заявки, просмотр фото в полном размере, смена статуса с вводом причины отклонения
Реестр объектов	Список ЖК с адресами, датами регистрации, бейджами количества дефектов, диалог добавления нового ЖК
Аналитический дашборд	Сводные виджеты, круговая диаграмма статусов, столбчатая диаграмма распределения дефектов по объектам и категориям

3.7. Роли и права доступа

Таблица 6 – Матрица прав доступа

Операция / Возможность	Анонимный заявитель	Менеджер объекта	Администратор
Подача новой заявки и загрузка файлов	Да	Да	Да
Просмотр статуса заявки по ID	Да	Да	Да
Просмотр полного списка всех заявок	Нет	Да	Да
Изменение статуса заявки («в работе», «выполнено», «отклонено»)	Нет	Да	Да
Просмотр аналитического дашборда	Нет	Да	Да
Создание и редактирование строительных объектов (ЖК)	Нет	Да	Да

Операция / Возможность	Анонимный заявитель	Менеджер объекта	Администратор
Удаление строительных объектов и заявок	Нет	Нет	Да
Управление учётными записями менеджеров	Нет	Нет	Да

4. ЛОГИЧЕСКАЯ МОДЕЛЬ И АРХИТЕКТУРА

4.1. Архитектура системы

Система построена на основе классической клиент-серверной архитектуры с разделением ответственности (Separation of Concerns) и Clean Architecture на бэкенде.

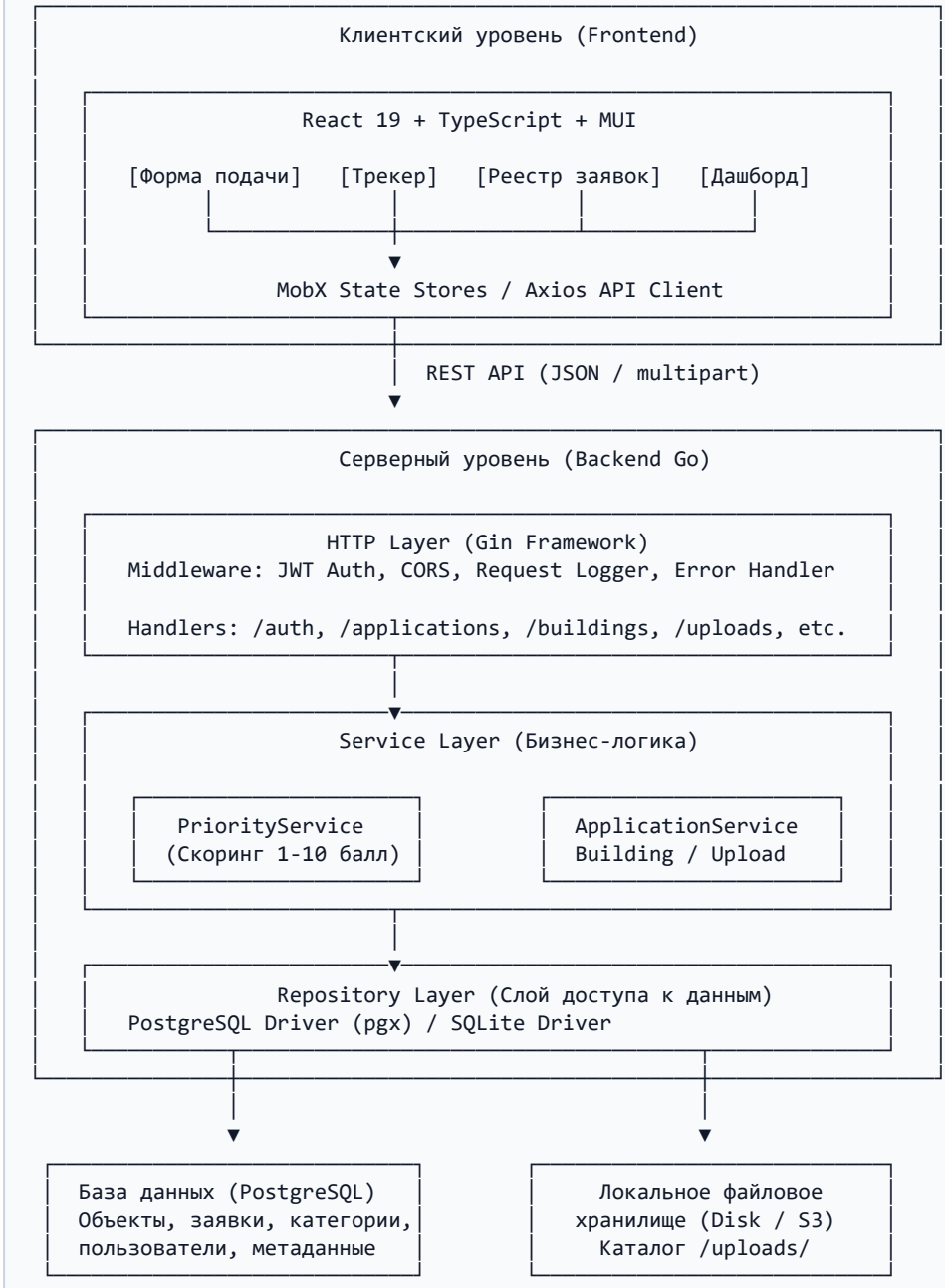


Рисунок 2 – Архитектура системы (диаграмма компонентов)

Таблица 7 – Программные компоненты

Компонент	Назначение
Frontend SPA (React)	Обеспечивает пользовательский интерфейс для подачи заявок, трекинга, реестра DataGrid и аналитических графиков
Gin HTTP Router & Handlers	Приём и валидация HTTP-запросов, маршрутизация, сериализация JSON, проверка JWT токенов
PriorityService	Инкапсулирует бизнес-правила расчёта приоритета заявки на основе категорий, описания, медиафайлов и истории
UploadHandler & Service	Приём бинарных файлов через multipart/form-data, проверка безопасности расширений, сохранение на диск и учёт метаданных
Repository Layer	Выполнение SQL-запросов, изоляция работы с БД PostgreSQL / SQLite от бизнес-логики сервисов
PostgreSQL 16	Реляционное хранилище постоянных структурированных данных приложения
Файловое хранилище	Каталог файловой системы сервера для надёжного хранения исходных фото и видеоматериалов

4.2. Алгоритм обработки входящей заявки

При поступлении запроса на создание заявки (`POST /api/applications`): 1. Валидация входных данных: проверка наличия обязательных полей (`name`, `description`, `email`, `building_id`); 2. Верификация переданных идентификаторов загруженных файлов (`upload_id`); 3. Передача параметров в `PriorityService`: - чтение базового веса категории из репозитория категорий; - анализ длины строки описания; - получение типов файлов из репозитория загрузок (проверка наличия видео); - проверка времени суток создания обращения; - выполнение запроса в БД на наличие аналогичных инцидентов по данному объекту; - проверка загруженности здания по числу дефектов; - расчёт скоринга с ограничением интервалом [1; 10]; 4. Сохранение заявки в БД со статусом `новый` и вычисленным значением `priority`; 5. Инкремент счётчика дефектов в паспорте соответствующего здания (`numberApplications = numberApplications + 1`); 6. Формирование ответа клиенту с возвратом присвоенного ID заявки.

4.3. Логическая модель данных (ER-модель)

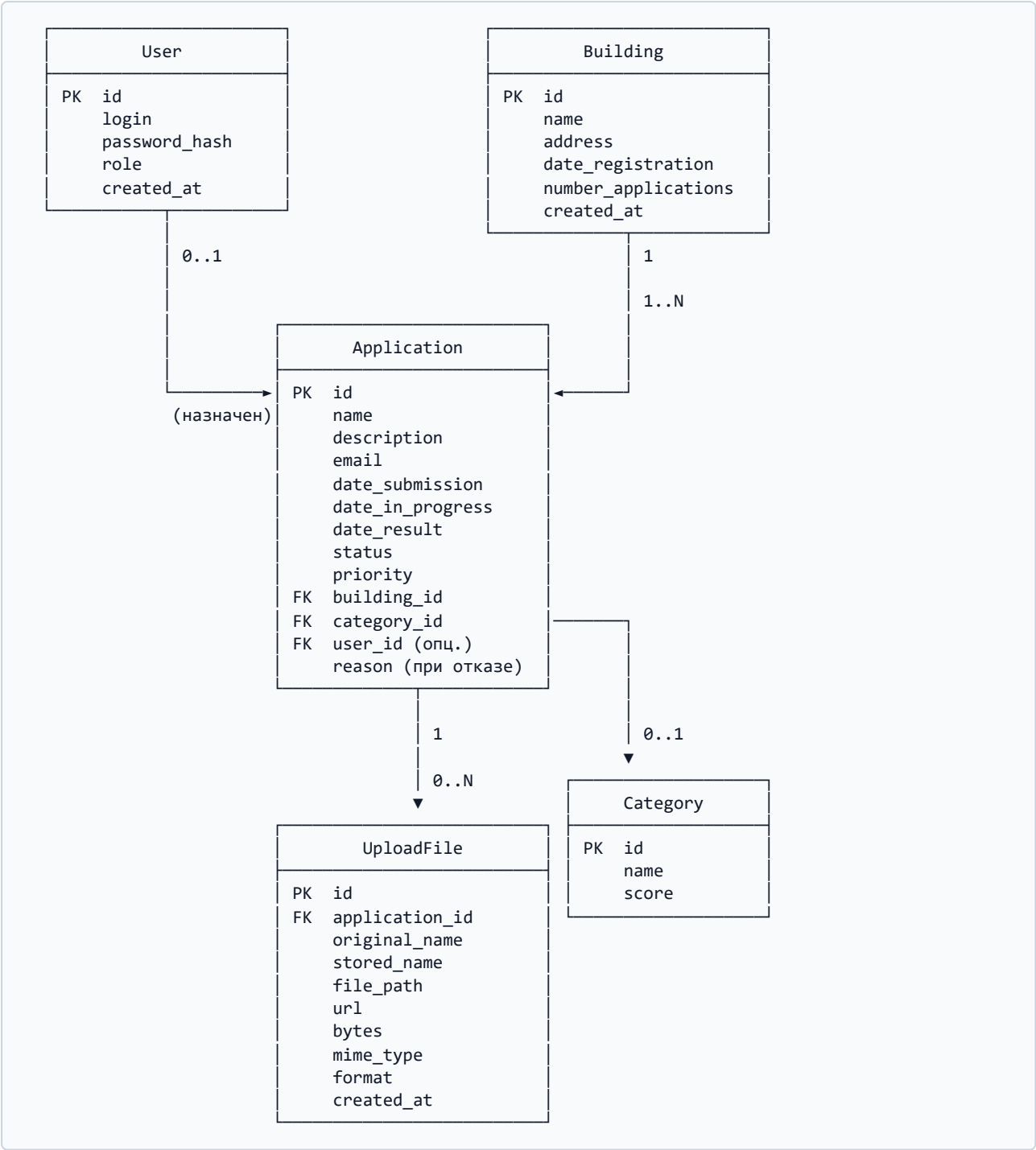


Рисунок 3 – Логическая модель данных (ER-диаграмма)

Таблица 8 – Описание сущностей базы данных

Сущность	Атрибуты	Описание
User	`id`, `login`, `password_hash`, `role`, `created_at`	Учётная запись сотрудника (менеджер, администратор)
Building	`id`, `name`, `address`, `date_registration`, `number_applications`, `created_at`	Паспорт строительного объекта (ЖК, здание)

Сущность	Атрибуты	Описание
Category	`id`, `name`, `score`	Справочник категорий дефектов с весовым коэффициентом опасности
Application	`id`, `name`, `description`, `email`, `date_submission`, `date_in_progress`, `date_result`, `status`, `priority`, `building_id`, `category_id`, `user_id`, `reason`	Карточка рекламации на дефект
UploadFile	`id`, `application_id`, `original_name`, `stored_name`, `file_path`, `url`, `bytes`, `mime_type`, `format`, `created_at`	Метаданные прикреплённого медиафайла

Таблица 9 – Связи между сущностями

Связь	Тип	Описание
Building → Application	1 : N	Одно здание может содержать множество зарегистрированных дефектов
Category → Application	1 : N	К одной категории дефекта относится множество конкретных заявок
Application → UploadFile	1 : N	К одной заявке может быть прикреплено произвольное количество файлов
User → Application	1 : N	Менеджер может быть назначен ответственным за исполнение ряда заявок

4.4. Бизнес-правила и ограничения целостности

Таблица 10 – Бизнес-правила (Business Rules)

№	Правило	Формализация / Логика
BR-1	Допустимые статусы заявки	Status ∈ {"новый", "в работе", "выполнено", "отклонено"}
BR-2	Фиксация даты взятия в работу	При переходе "новый" → "в работе": date_in_progress = now()
BR-3	Фиксация даты завершения	При переходе "в работе" → "выполнено": date_result = now()
BR-4	Обязательность причины при отклонении	При Status = "отклонено": reason ≠ NULL и len(reason) ≥ 5
BR-5	Инкремент счётчика здания	При создании заявки: Building.numberApplications += 1
BR-6	Диапазон скоринга приоритета	Для любой заявки: 1 ≤ priority ≤ 10

№	Правило	Формализация / Логика
BR-7	Уникальность логина пользователя	UNIQUE(User.login)
BR-8	Контроль допустимых типов файлов	MimeType ∈ {image/jpeg, image/png, image/webp, video/mp4, video/quicktime, application/pdf}
BR-9	Каскадность метаданных файлов	Удаление заявки ⇒ каскадное удаление связанных записей в UploadFile

---

5. ТРЕБОВАНИЯ К ПРОЦЕССУ РАЗРАБОТКИ

5.1. Методология и организация работ

- Разработка ведётся по гибкой итеративной методологии (Agile/Scrum) двухнедельными спринтами;
- Контроль версий осуществляется в распределённой системе **Git**;
- Используется модель ветвления: основная ветка `main` / `master`, разработка функционала в отдельных ветках `feature/<наименование>`, интеграция через обязательные Pull Request;
- Обязательное проведение взаимного рецензирования кода (Code Review);
- Автоматизированный конвейер CI/CD обеспечивает сборку, линтинг и запуск тестов при каждом push-событии.

5.2. Требования к качеству кода

- **Backend:** строгое соблюдение рекомендаций Effective Go, форматирование утилитой `gofmt`, статический анализ через `golangci-lint`, покрытие модульными тестами сервиса скоринга приоритета не менее 80%;
- **Frontend:** использование строгого режима TypeScript (`strict: true`), линтинг через `ESLint` с конфигурацией React Hooks, соблюдение структуры FSD;
- Обработка всех ошибок (`err != nil`) в Go без скрытого игнорирования;
- Проверка зависимостей на уязвимости (`go audit` / `npm audit`).

5.3. ИИ-ассистирование при разработке

В процессе проектирования, реализации и тестирования системы допускается применение ИИ-ассистентов (Antigravity, Claude, ChatGPT, GitHub Copilot) для форсирования инженерной продуктивности. Ответственность за архитектурные решения, информационную безопасность и работоспособность кода в полном объёме несёт разработчик.

Таблица 11 – Области применения ИИ-ассистентов

Область	Роль ИИ-ассистента	Контроль человеком
Аналитика и ТЗ	Помощь в структурировании спецификации, формулирование критериев приёмки	Согласование с заказчиком и преподавателем
Проектирование	Черновики схем БД, структур DTO и OpenAPI спецификаций	Экспертиза архитектора
Генерация кода	Написание типовых CRUD-хэндлеров, валидаторов, мапперов данных	Обязательный Code Review и тестирование
Тестирование	Генерация наборов входных параметров для тестирования алгоритма скоринга	Проверка корректности тест-кейсов
Рефакторинг	Оптимизация SQL-запросов и структуры MobX хранилищ	Сравнение до/после, регрессионный прогон
Документация	Черновики README, инструкций пользователя и API-документации	Проверка фактической достоверности

Правила безопасного использования ИИ-ассистентов: 1. Категорический запрет на передачу в сторонние облачные нейросети конфиденциальных сведений, закрытых паролей, боевых токенов и персональных данных заявителей; 2. Каждый сгенерированный фрагмент кода подвергается полному пониманию и ручной проверке разработчиком; 3. Лицензионная чистота: недопущение использования фрагментов кода под проприетарными лицензиями; 4. Ожидаемый эффект: сокращение трудоёмкости создания типовых компонентов на 15–20%.

5.4. Окружения

- **Development (локальное):** ОС Windows/Linux/macOS, запуск Go API локально или через Docker, SQLite или локальный PostgreSQL, Webpack/Vite dev-сервер React;
- **Staging / Testing:** Docker Compose стенд с изолированной сетью, экземпляром PostgreSQL 16 и моковыми данными объектов;
- **Production:** Защищённый Linux VPS/VDS сервер с Nginx Reverse Proxy (SSL/TLS терминация), Docker Compose контейнеры фронтенда, бэкенда и СУБД.

---

6. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

Состав программной документации формируется в соответствии с требованиями Единой системы программной документации (ЕСПД): **ГОСТ 19.101-77**, **ГОСТ 19.201-78** (Техническое задание), **ГОСТ 19.301-79** (Программа и методика испытаний), **ГОСТ 19.505-79** (Руководство оператора).

Таблица 12 – Состав программной документации

Документ	Содержание	Этап разработки
Техническое задание	Требования к системе, функционалу, архитектуре и смете (настоящий документ)	1
Описание архитектуры и схемы БД	Диаграммы компонентов, ER-модель, спецификация таблиц и индексов	2
Спецификация REST API	Интерактивная спецификация в формате OpenAPI 3.1 (Swagger)	2–3
Программа и методика испытаний (ПМИ)	Перечень контрольных проверок, тест-кейсов и критериев прохождения	6
Отчёт о результатах испытаний	Протоколы функционального, нагрузочного и интеграционного тестирования	6
Руководство администратора	Развёртывание в Docker Compose, конфигурация окружения, резервное копирование	7–8
Руководство пользователя / менеджера	Инструкция по приёму заявок, трекингу, изменению статусов и просмотру аналитики	8
Исходный код и файл README.md	Структура монорепозитория, инструкция по сборке, локальному запуску и сидам	1–8

7. ЭТАПЫ РАЗРАБОТКИ

Таблица 13 – Этапы и результаты работ

№	Этап	Содержание работ	Результат
1	**Аналитика и проектирование**	Анализ требований предметной области строительного контроля, формулирование ТЗ, прототипирование UI/UX	Согласованное ТЗ, кликабельные макеты экранов
2	**Проектирование архитектуры и БД**	Разработка ER-модели, спецификации REST API, подготовка SQL-миграций и начальных сидов	SQL-схема БД, контракт OpenAPI 3.1
3	**Разработка Backend (Go)**	Реализация Gin-роутера, JWT авторизации, CRUD сущностей, слоя репозиториев PostgreSQL/SQLite	Работающее серверное ядро API
4	**Алгоритмический модуль скоринга**	Реализация и покрытие тестами PriorityService (многофакторный расчёт приоритета 1–10)	Протестированный модуль скоринга

№	Этап	Содержание работ	Результат
5	**Файловый сервис и хранилище**	Multipart-обработка, валидация типов/размеров, генерация URL, статическая раздача `/api/uploads`	Модуль работы с медиафайлами
6	**Разработка Frontend (React FSD)**	Разработка публичной формы подачи заявки, трекера по ID, реестра заявок DataGrid и карточек ЖК	Полнофункциональный клиентский интерфейс
7	**Аналитический модуль и дашборд**	Бэкенд агрегация статистики, визуализация графиков и диаграмм на фронтенде	Экран аналитики дефектов
8	**Тестирование, Docker и сдача**	Сквозные E2E тесты, сборка Dockerfile/docker-compose, оформление руководств и сдача заказчику	Готовая к промышленной эксплуатации система

---

8. СРОКИ РЕАЛИЗАЦИИ

Общий плановый срок реализации проекта составляет **16 календарных недель** (около 4 месяцев) при согласованном составе команды. Разработка клиентской и серверной частей ведётся параллельно после фиксации контракта API на 4-й неделе.

Таблица 14 – Календарный план разработки (недели проекта)

Этап / Неделя	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1. Аналитика и требования	■	■														
2. Проектирование БД и API			■	■												
3. Backend (Go ядро и CRUD)					■	■	■	■								
4. Модуль скоринга приоритета						■	■									
5. Frontend: форма и трекер					■	■	■									
6. Frontend: DataGrid и статусы								■	■	■						
7. Файлы и аналитика									■	■	■					
8. Интеграционное тестирование												■	■			
9. Документация и Docker													■	■	■	
10. Приёмо-сдаточные испытания																■

Таблица 15 – Контрольные точки проекта (вехи)

Веха	Срок	Критерий достижения
М1 – ТЗ и архитектура согласованы	Конец нед. 4	Утверждено ТЗ, спроектирована схема БД, зафиксирован OpenAPI-контракт
М2 – MVP Backend готов	Конец нед. 8	Реализован REST API, работают миграции, функционирует алгоритм скоринга

Веха	Срок	Критерий достижения
М3 – MVP Frontend готов	Конец нед. 10	Работает подача заявки жильцом, просмотр списка заявок менеджером
М4 – Интегрированная бета-версия	Конец нед. 13	Реализован полный цикл смены статусов, аналитика, загрузка медиа
М5 – Релиз и ввод в эксплуатацию	Конец нед. 16	Успешное прохождение приёмочных испытаний, развёртывание в Docker

Риски по срокам: задержка согласования категорий дефектов, изменение требований к скорингу, сетевые ограничения окружения. Предусмотрен буферный резерв в объёме 2 недель.

---

9. СОСТАВ КОМАНДЫ ПРОЕКТА

Таблица 16 – Роли, загрузка и трудоёмкость

Роль	Кол-во	Загрузка (FTE)	Часы	Зона ответственности
Руководитель проекта (PM)	1	0,3	192	Планирование, контроль сроков, взаимодействие с заказчиком
Системный аналитик	1	0,5	320	Разработка ТЗ, бизнес-правила скоринга, схема БД, макеты
Backend-разработчик (Go)	2	1,0	1 280	Серверная архитектура, REST API, PriorityService, БД, тесты
Frontend-разработчик (React)	1	1,0	640	SPA интерфейс, FSD архитектура, MobX, интеграция с API
DevOps-инженер	1	0,3	192	Docker Compose, конфигурация PostgreSQL, CI/CD, Nginx
QA-инженер (тестировщик)	1	0,7	448	Тест-планы, ручное и автоматизированное тестирование, нагрузка
Технический писатель	1	0,2	128	Руководства оператора и администратора, оформление документации
ИТОГО	**7**	**≈ 5,0**	**3 200**	Полный цикл проектирования, разработки и внедрения

Примечание: **Для учебного/инициативного формата реализации допускается совмещение ролей студентом-разработчиком (Fullstack Go + React, аналитика, DevOps) при консультационной поддержке научного руководителя.**

---

10. ОРИЕНТИРОВОЧНАЯ СТОИМОСТЬ РАЗРАБОТКИ

Оценка бюджета выполнена методом калькуляции трудозатрат «снизу вверх» на основе рыночных ставок профильных IT-специалистов.

Таблица 17 – Фонд оплаты труда (ФОТ)

Роль	Трудоёмкость, ч	Ставка, ₽/ч	Стоимость, ₽
Руководитель проекта	192	3 000	576 000
Системный аналитик	320	2 500	800 000
Backend-разработчики (Go, 2 чел.)	1 280	3 000	3 840 000
Frontend-разработчик (React)	640	2 800	1 792 000
DevOps-инженер	192	3 000	576 000
QA-инженер	448	2 000	896 000
Технический писатель	128	1 500	192 000
Итого ФОТ	**3 200**	—	**8 672 000**

Таблица 18 – Общая смета проекта

Статья затрат	Сумма, ₽	Примечание
Фонд оплаты труда	8 672 000	Базовые трудозатраты специалистов команды
Налоги и страховые взносы (30%)	2 601 600	Социальные и пенсионные отчисления
Серверная инфраструктура (1 год)	120 000	Аренда облачного сервера (VPS), бэкапы, домен
Лицензии и стороннее ПО	60 000	Инструменты мониторинга, сертификаты
Резерв на непредвиденные риски (10%)	1 145 360	Поправка на изменение требований и интеграционные риски
ИТОГО ПО ПРОЕКТУ	**≈ 12 598 960**	**Ориентировочно 12,6 млн рублей**

Экономический эффект ИИ-ассистирования: При регулярном использовании современных ИИ-ассистентов (согласно регламенту п. 5.3) экономия трудозатрат разработчиков составляет до 18%, что позволяет сберечь от 1,0 до 1,3 млн рублей проектного бюджета.

Таблица 19 – Стоимость совокупного владения после запуска (TCO в год)

Статья расходов	Сумма, ₽/год
Техническая поддержка и сопровождение (2-я линия)	1 800 000

Статья расходов	Сумма, ₽/год
Инфраструктура и облачный хостинг	120 000
Плановые функциональные доработки и обновления	600 000
Итого владение в год	**≈ 2 520 000**

---

11. ТРЕБОВАНИЯ К ПОРЯДКУ КОНТРОЛЯ И ПРИЁМКИ

11.1. Виды испытаний

1. **Модульное и интеграционное тестирование:** автоматический запуск тестов в CI конвейере;
2. **Функциональные испытания:** сквозная проверка сценариев по Программе и методике испытаний (ПМИ);
3. **Нагрузочное тестирование:** симуляция 100 одновременных запросов на создание заявок с прикреплением файлов;
4. **Проверка информационной безопасности:** тестирование защиты закрытых маршрутов без JWT, проверка хэширования паролей, попытка инъекций;
5. **Опытная эксплуатация:** пилотное использование в течение 2 недель сотрудниками управляющей организации на реальных объектах.

11.2. Порядок приёмки

1. Развёртывание релизного билда в демонстрационном стенде;
2. Выполнение приёмо-сдаточных испытаний по чек-листу в присутствии представителей Заказчика;
3. Передача исходных текстов программы в репозитории Git;
4. Передача комплекта программной документации в форматах PDF и DOCX;
5. Подписание двустороннего Акта приёмки-передачи системы в опытную эксплуатацию.

11.3. Критерии приёмки

Таблица 20 – Чек-лист приёмо-сдаточных испытаний

№	Проверяемый функционал	Ожидаемый результат
1	Подача заявки жильцом	Заявка успешно сохраняется в БД со статусом «новый», возвращается валидный ID
2	Загрузка медиафайлов	Фото и видео сохраняются на сервере, доступны для просмотра по сгенерированному URL

№	Проверяемый функционал	Ожидаемый результат
3	Расчёт приоритета (скоринг)	Для заявки с видео, длинным описанием и аварийной категорией выставляется приоритет 8–10
4	Авторизация персонала	Вход по логину/паролю возвращает JWT токен; неверный пароль отсекается с кодом 401
5	Реестр заявок в DataGrid	Менеджер видит заявки с цветовой индикацией приоритета, работает пагинация и фильтрация
6	Смена статуса на «в работе»	Статус обновляется, автоматически фиксируется текущий таймстемп `dateInProgress`
7	Отклонение заявки	Попытка отклонить без указания причины блокируется; при вводе причины статус меняется на «отклонено»
8	Паспорт объектов (ЖК)	Счётчик `numberApplications` увеличивается при создании новой заявки по данному ЖК
9	Аналитический дашборд	Виджеты корректно отражают агрегированные количества по статусам и проблемным зданиям
10	Защита маршрутов	Обращение к эндпоинтам менеджера без Bearer-токена возвращает статус 401 Unauthorized

11.4. Передаваемые материалы

- Исходный код бэкенда на Go и фронтенда на React в репозитории Git с историей коммитов;
- Конфигурационные файлы сборки (`Dockerfile` , `docker-compose.yml` , `.env.example`);
- Инициализационные SQL-скрипты миграций и наполнения начальными справочниками (сиды);
- Комплект документации по разделу 6 (ТЗ, Руководство оператора, Руководство администратора, OpenAPI спецификация);
- Акт приёмки-передачи программного обеспечения.

ПРИЛОЖЕНИЕ А. ПРИНЯТЫЕ ДОПУЩЕНИЯ

Таблица А.1 – Принятые проектные допущения

Вопрос / Параметр	Принятое инженерное решение
Аутентификация заявителей	Анонимная подача без регистрации для максимального снижения барьера входа жильцов; трекинг осуществляется по уникальному ID заявки

Вопрос / Параметр	Принятое инженерное решение
Хранилище файлов	Для MVP используется локальный том сервера (`/uploads`), с абстракцией интерфейса, позволяющей бесшовно переключиться на S3-совместимое хранилище (MinIO / AWS S3)
Шкала приоритета	Целочисленная шкала от 1 (минимальный) до 10 (наивысший/аварийный), понятная персоналу диспетчерской службы
Форматы видео	Поддержка web-форматов MP4 и WebM с ограничением длительности до 60 секунд
База данных	Полная совместимость схемы с PostgreSQL 16 для production и SQLite для автономного локального запуска без внешних сервисов
Уведомления заявителей	В базовой версии — информирование через web-трекер; отправка email и Telegram-уведомлений вынесена в план второй очереди развития